ESP Lab2: Prep Sheet

Multirate FIR Branching Filter

November 7, 2026

Group / Team	**Team member 1:**	Pham, Tuong Minh
ESPL/01 /	**Team member 2:**	Ul Haq, Waleed

Table 1: Filter specifications of lab group ESPL/01

Parameter		**ESPL/01**
Sampling rate of ADC and DAC		$f_s = 50\,\text{kHz}$
Lowpass filter	passband edge stopband edge	$f_{pass,LP} = 3100\,\text{Hz}$ $f_{stop,LP} = 3350\,\text{Hz}$
Complementary highpass filter	stopband edge passband edge	$f_{stop,HP} = 3100\,\text{Hz}$ $f_{pass,HP} = 3350\,\text{Hz}$
Total passband ripple		$\delta_{pass} = 0.01$ (linear, not dB)
Minimum stopband attenuation		$\delta_{stop} = 40\,\text{dB}$
Quantisation of signals and coefficients		16 bit (1Q15 format)

3 Preparation

Use of generative AI

Tool: Claude (Anthropic). The lab instructions and the Moodle files were provided as context. Essential prompts (translated from German, condensed):

1. "Create an empty LaTeX document for the preparation sheet of ESP Lab 2."
2. "Section 3.1: Explain to me how the filter order of the single-rate solution is obtained from the MATLAB output of `dec_kernel_int_WS20_EMIL.m`, how many DSP cycles are available per sampling period and whether a real-time implementation is feasible."
3. "Section 3.2: Explain to me how M_{opt} is obtained with `roots_demo.m` and how the edge frequencies and passband ripples of the decimation, interpolation and kernel filter are determined. Extend `dec_kernel_int_WS20_EMIL.m` as simply as possible with the filter design using `firpmord()`/`firpm()`, the magnitude plots and the export of the polyphase coefficients."
4. "Explain to me why the cascade plot in MATLAB shows a higher stopband attenuation than a measured magnitude response of a multirate filter."
5. "Section 3.3: Explain to me how the computational effort and the group delay of the single-rate and the multirate filter are calculated using the formulas from the lecture, and write the results as LaTeX text for the prep sheet."

3.1 Conventional lowpass filter

Design a linear-phase FIR low-pass and its complementary high-pass using firpmord() and firpm(). Use the specification above und the script dec_kernel_int.m provided in Moodle. Plot both magnitude responses in dB for both filters in two different subplots. Verify δ_{pass} and δ_{stop}.

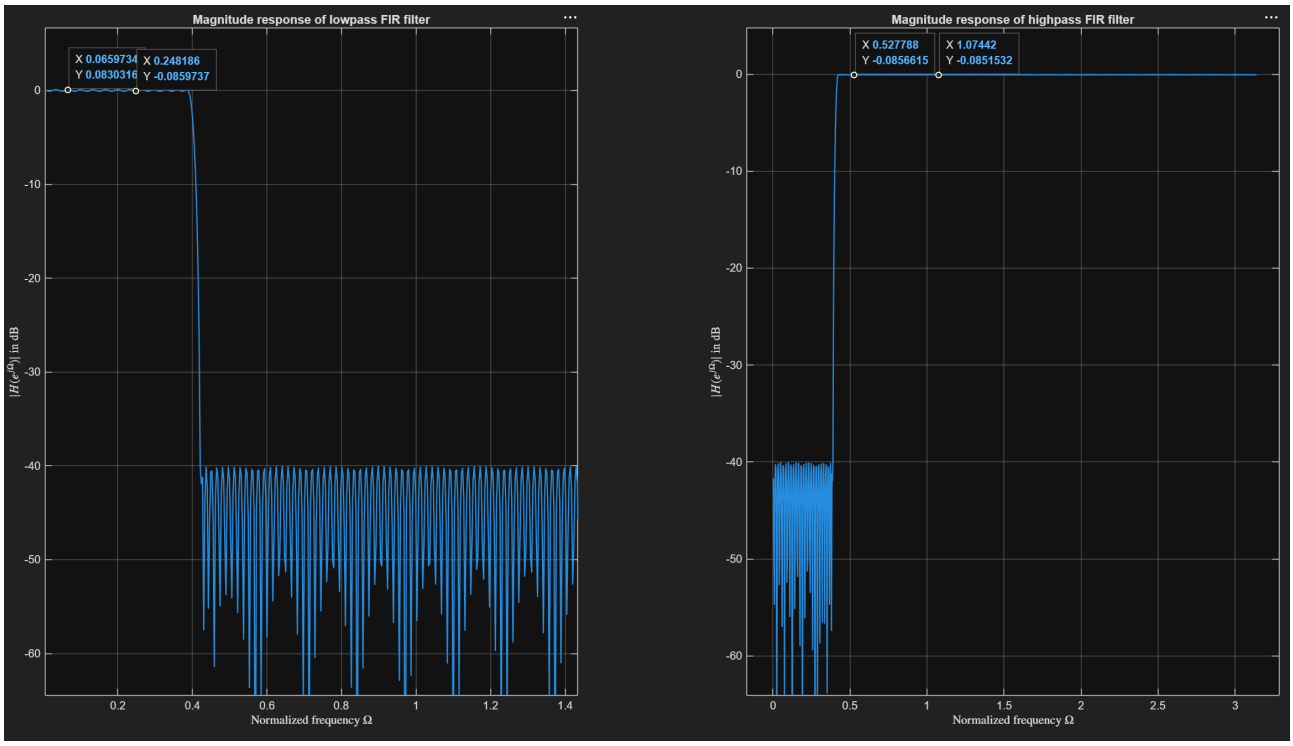


Figure 1: Magnitude responses of the single-rate FIR low-pass (left) and the complementary high-pass (right), designed with `dec_kernel_int_WS20_EMIL.m`, $N = 391$.

Verification. Fig. 1 shows a stopband attenuation of at least 40 dB for both filters, so δ_{stop} is met. The maximum passband deviation read from the plot is about ± 0.085 dB. Converted to a linear value:

$$10^{0.085/20} \approx 1.0098, \qquad 10^{-0.085/20} \approx 0.9902 \quad \Rightarrow \quad \big|\,|H| - 1\,\big| \approx 0.0098 \leq \delta_{pass} = 0.01.$$

The passband ripple therefore also meets the requirement. For comparison, $\delta_{pass} = 0.01$ corresponds to $20 \log_{10}(1.01) \approx 0.086$ dB.

From the Matlab output, record the filter degree (order) for the single-rate solution.

Filter order. `firpmord()` estimates an order of 389; the script adds 2 as a safety margin, so the single-rate filter has the order $N = 391$ (MATLAB output: `N_FIR=391`). An FIR filter of order N computes $y[n] = \sum_{k=0}^{N} b_k\, x[n-k]$, so it has $N + 1 = 392$ coefficients. The high order is caused by the narrow transition band of only 250 Hz compared to the sampling frequency ($\Delta f / f_s = 0.005$).

Compute the DSP cycles available per sampling period at $f_s = 50$kHz and $f_{DSP} = 456$MHz. Discuss the feasibility of real-time implementation on the DSP without further code optimization.

At $f_{DSP} = 456$ MHz and $f_s = 50$ kHz, the number of DSP cycles available per sampling period is

$$\frac{f_{DSP}}{f_s} = \frac{456\,\text{MHz}}{50\,\text{kHz}} = 9120 \text{ cycles}.$$

The single-rate filter has 392 coefficients, i.e. 392 multiply-accumulate operations per sample plus the update of the delay line. With the optimised assembly routine `FIR_filter_asm_sc.asm` ($16 + 6N$ cycles), the filter requires about $16 + 6 \cdot 392 \approx 2370$ cycles, i.e. about 26 % of the available time, so a real-time implementation is feasible. Without code optimisation (plain C), several cycles are needed

per coefficient for loading, multiplying, accumulating and shifting the delay line. With an estimated 15 to 25 cycles per coefficient, the filter would take about 65 % to more than 100 % of the available cycles, which is at the limit of real-time operation. This high effort, caused by the narrow transition band, motivates the multirate implementation in Section 3.2.

3.2 Multi-rate FIR filter design

3.2.1 Optimum choice of downsampling/upsampling factor M

Add the exact value for M_{opt} and the nearest integer number to the prep sheet.

Listing 1: `roots_demo.m` with the specification of ESPL/01, Section 3.2.1

```
% roots_demo.m
%
% US 07-Nov-23
%
% demo for finding M_opt
clear
close all
Fs = 50000;
fpass = 3100;
fstop = 3350;

a = fstop^2 - fpass^2;
b = -(fpass + fstop)^2;
c = 2 * Fs  * (fpass   + fstop);
d = -Fs^2;

my_polynom = [a b c d]
roots(my_polynom)

```

Listing 2: MATLAB command window output, Section 3.2.1

```
my_polynom =

1.0e+09 *

0.0016   -0.0416    0.6450   -2.5000

ans =

10.2352 +13.6434i
10.2352 -13.6434i
5.3296 + 0.0000i
```

Result. The cubic polynomial has one real root and a pair of complex conjugate roots. Only the real root is a valid down-/upsampling factor, so the optimum factor is

$$M_{opt} = 5.3296 \approx 5 \quad \text{(nearest integer)}.$$

For the lab tasks, $M = L = 4$ is used as specified in Table 2 of the assignment.

3.2.2 Decomposition into decimation, interpolation and kernel filter

Determine the passband and stopband edge frequencies of the decimation and the kernel filter and the passband ripple of both filters. Extend `dec_kernel_int.m` with the design of the FIR filters via `firpmord()` and `firpm()`, plot the magnitude responses of the decimation filter, the kernel filter and the cascade of all three filters, and write the coefficient sets to the header file. Explain the approach and the verification of the results.

Filter parameters. With $M = L = 4$, the low sampling frequency is $f_s' = f_s/M = 12.5\,\text{kHz}$. The kernel filter runs at f_s' and realises the steep transition of the original low-pass filter. The decimation and interpolation filters run at f_s and are identical.

Table 2: Theoretical filter parameters for $M = L = 4$

Filter	**Sampling rate**	f_{pass}	f_{stop}	δ_{pass}	**Stopband att.**
Decimation / interpolation	$f_s = 50\,\text{kHz}$	3100 Hz	$f_s' - f_{stop,LP} = 9150\,\text{Hz}$	0.01/3	40 dB
Kernel	$f_s' = 12.5\,\text{kHz}$	3100 Hz	3350 Hz	0.01/3	40 dB

The kernel filter keeps the passband and stopband edges of the original filter, $f_{pass} = 3100\,\text{Hz}$ and $f_{stop} = 3350\,\text{Hz}$.

The decimation filter has the same passband edge of 3100 Hz, so that the wanted signal passes unchanged. Its stopband edge only has to prevent aliasing into the range below $f_{stop,LP}$: after downsampling, a component at f appears at $f_s' - f$. All components between $f_{stop,LP}$ and $f_s' - f_{stop,LP}$ are therefore folded into the stopband of the kernel filter and are removed there. Only components above $f_s' - f_{stop,LP} = 12.5\,\text{kHz} - 3.35\,\text{kHz} = 9.15\,\text{kHz}$ would fall below $f_{stop,LP}$ and must be suppressed by the decimation filter. The same holds for the interpolation filter: the images of the kernel output (0 to 3350 Hz) start at $f_s' - f_{stop,LP} = 9.15\,\text{kHz}$. This wide transition band (3100 Hz to 9150 Hz) allows a short decimation and interpolation filter.

The signal passes through three filters in cascade (decimation, kernel and interpolation filter). In the worst case, their passband ripples add up. Each filter is therefore designed with one third of the total ripple, $\delta_{pass} = 0.01/3 \approx 0.0033$. The minimum stopband attenuation of 40 dB is kept for each filter.

MATLAB implementation. Listing 3 shows the code added to `dec_kernel_int_WS20_EMIL.m`. Both filters are designed with `firpmord()` and `firpm()`, including the safety margin of 2 as for the single-rate filter. The MATLAB output is `N_FIR_Dec_Int = 20` and `N_FIR_kernel = 116`. The decimation filter is split into $M = 4$ polyphase filters with the coefficients $b_{Dec}[kM + \lambda]$, $\lambda = 0, \ldots, 3$; the interpolation filter uses the same coefficient sets with its own delay lines. All coefficient sets are written to `dec_kernel_int_WS20_EMIL.h` with `write_coeff.m`.

Listing 3: Design of the decimation/interpolation and kernel filter, magnitude plots and header export, added to `dec_kernel_int_WS20_EMIL.m`, Section 3.2.2

```
        % ---- 3.2 multirate filter design ----
        % optimum factor (see roots_demo.m) and factor for the lab (Table 2)
        r = roots([fstop^2-fpass^2, -(fpass+fstop)^2, 2*Fs*(fpass+fstop), -Fs^2]);
        MMmin = real(r(abs(imag(r)) < 1e-6));
        MM = 4;
        Fs_low = Fs/MM;

        % decimation / interpolation filter (at Fs), ripple 1/3 each
        fpass_dec = fpass;                % 3100 Hz
        fstop_dec = Fs_low - fstop;       % 12500 - 3350 = 9150 Hz
        [N_FIR_Dec_Int,fo,mo,w] = firpmord([fpass_dec fstop_dec], [1 0], [delta_pass/3 delta_stop], Fs
    );
        N_FIR_Dec_Int = N_FIR_Dec_Int + 2;
        b_dec = firpm(N_FIR_Dec_Int,fo,mo,w);

        % kernel filter (at Fs/M), ripple 1/3
        [N_FIR_kernel_MM,fo,mo,w] = firpmord([fpass fstop], [1 0], [delta_pass/3 delta_stop], Fs_low);
        N_FIR_kernel_MM = N_FIR_kernel_MM + 2;
        b_kernel = firpm(N_FIR_kernel_MM,fo,mo,w);

        fprintf('\n M_opt = %.4f, M = %d, N_FIR_Dec_Int = %d, N_FIR_kernel = %d\n\n', ...
        MMmin, MM, N_FIR_Dec_Int, N_FIR_kernel_MM);

        % magnitude responses at Fs
        f = linspace(0, Fs/2, 4000);
        H_dec = freqz(b_dec, 1, f, Fs);
        H_ker = freqz(b_kernel, 1, f, Fs_low);   % kernel runs at Fs/M -> periodic with Fs/M
```

```
27
28      figure;                                  % decimation and kernel filter in one plot
29      plot(f/1e3, db(H_dec)), hold on
30      plot(f/1e3, db(H_ker)), grid
31      axis([0 Fs/2e3 -80 5]);
32      legend('Decimation / interpolation filter', 'Kernel filter (at f_s/M, periodic)');
33      title('Magnitude response (dB)'); xlabel('f in kHz'); ylabel('|H| in dB');
34
35      figure;                                  % cascade of the three filters
36      plot(f/1e3, db(H_dec.*H_ker.*H_dec)), grid
37      title('Cascade: decimation, kernel, interpolation filter'); xlabel('f in kHz');
38
39      % ---- header file (inserted before the last fclose) ----
40      % polyphase filters of the decimation filter (also used for interpolation)
41      N_poly = ceil(length(b_dec)/MM);
42      fprintf(file_ID, '#define N_delays_poly %d\n\n', N_poly);
43      for lambda = 0:MM-1
44      fprintf(file_ID, 'short H_dec_%d_delays[N_delays_poly];\n', lambda);
45      fprintf(file_ID, 'short H_int_%d_delays[N_delays_poly];\n', lambda);
46      write_coeff(file_ID, sprintf('b_poly_%d', lambda), b_dec(lambda+1:MM:end), N_poly);
47      end
48
49      % kernel filter
50      fprintf(file_ID, '#define N_delays_kernel %d\n', length(b_kernel));
51      fprintf(file_ID, 'short H_kernel_delays[N_delays_kernel];\n');
52      write_coeff(file_ID, 'b_kernel', b_kernel, length(b_kernel));
53
```

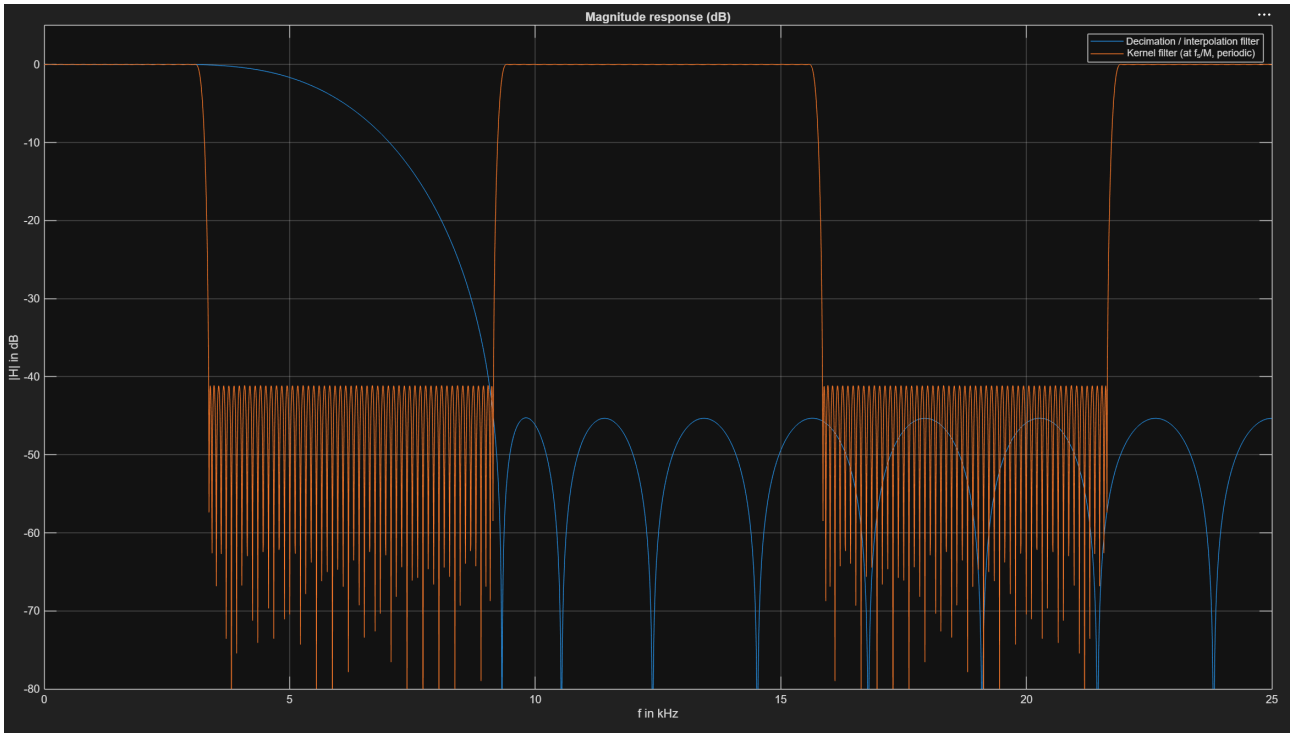


Figure 2: Magnitude responses of the decimation/interpolation filter (at f_s) and of the kernel filter (at f_s/M, periodic with $f_s' = 12.5\,\mathrm{kHz}$).

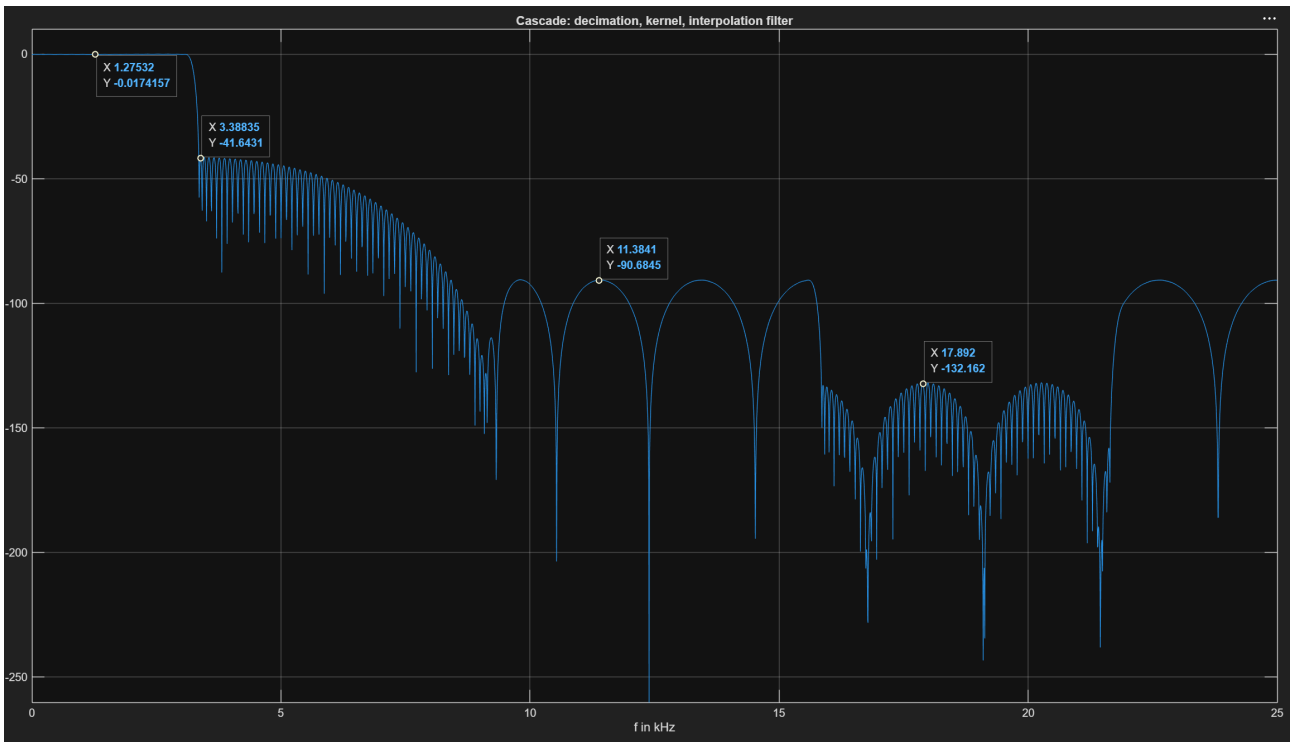


Figure 3: Magnitude response of the cascade of decimation, kernel and interpolation filter.

Verification. Fig. 2 acts as a sketch of the spectra. Because the kernel filter runs at $f_s' = 12.5\,\text{kHz}$, its passband repeats at multiples of f_s', i.e. at 9.4 to 15.6 kHz and above 21.9 kHz. All of these repetitions lie in the stopband of the decimation/interpolation filter, which starts at 9.15 kHz. The decimation filter therefore prevents aliasing into the kernel passband, and the interpolation filter removes the corresponding images. In the frequency range between 3.35 kHz and 9.15 kHz, the decimation filter does not attenuate, but here the kernel filter provides at least 40 dB.

The cascade in Fig. 3 meets the specification: the passband deviation is below 0.02 dB and the stopband attenuation is at least 41.6 dB (at 3.39 kHz). Note that this cascade response $H_{Dec}(f)\,H_{Kernel}(f)\,H_{Int}(f)$ neglects aliasing: input components near multiples of f_s' are folded into the passband by downsampling and are then only attenuated once by the stopband of the decimation filter. In a measurement, the attenuation in these regions is therefore about 45 dB instead of about 90 dB shown in the plot, which still meets the specification.

Listing 4: Generated header file `dec_kernel_int_WS20_EMIL.h` with the polyphase coefficients of the decimation/interpolation filter and the kernel coefficients, Section 3.2.2

```
//-----------------------------------------
// designed with -- dec_kernel_int_WS20_EMIL.m --
// Fs = 50000.00
// MMmin =   5.33
// fstop = 3350.00
// fpass = 3100.00
// delta_pass =   0.01
// delta_stop_dB =  40.00
// N_FIR = 391, instead two stages with N_FIR_Dec_Int = 20, N_FIR_KERNEL = 116
//-----------------------------------------
#define N_delays_poly 6
short H_dec_0_delays[N_delays_poly];
short H_int_0_delays[N_delays_poly];
short b_poly_0[6]={
    218,  -1026,   4930,   4930,  -1026,    218,
};
short H_dec_1_delays[N_delays_poly];
short H_int_1_delays[N_delays_poly];
short b_poly_1[6]={
```

```
    248,  -1086,   7339,   2101,   -462,      0,
};
short H_dec_2_delays[N_delays_poly];
short H_int_2_delays[N_delays_poly];
short b_poly_2[6]={
    51,    -87,   8286,    -87,     51,      0,
};
short H_dec_3_delays[N_delays_poly];
short H_int_3_delays[N_delays_poly];
short b_poly_3[6]={
    -462,   2101,   7339,  -1086,    248,      0,
};
#define N_delays_kernel 117
short H_kernel_delays[N_delays_kernel];
short b_kernel[117]={
    63,   -104,     -1,     39,    -15,    -55,
    12,     56,    -23,    -64,     32,     69,
    -46,    -74,     62,     77,    -80,    -78,
    101,     77,   -124,    -72,    149,     63,
    -176,    -50,    205,     32,   -235,     -8,
    267,    -22,   -298,     61,    330,   -108,
    -362,    166,    394,   -236,   -424,    323,
    452,   -430,   -478,    565,    502,   -742,
    -523,    986,    541,  -1354,   -555,   1987,
    565,  -3417,   -571,  10410,  16957,  10410,
    -571,  -3417,    565,   1987,   -555,  -1354,
    541,    986,   -523,   -742,    502,    565,
    -478,   -430,    452,    323,   -424,   -236,
    394,    166,   -362,   -108,    330,     61,
    -298,    -22,    267,     -8,   -235,     32,
    205,    -50,   -176,     63,    149,    -72,
    -124,     77,    101,    -78,    -80,     77,
    62,    -74,    -46,     69,     32,    -64,
    -23,     56,     12,    -55,    -15,     39,
    -1,   -104,     63,};
```

3.3 Comparison of computational effort

Compare the computational effort $E_{total,single}$ of the original filter and $E_{total,multi}$ of the multirate filter for $M = L = 4$: order of the decimation/interpolation filter, order of the kernel filter, computational effort. How probable is a successful real-time implementation? Determine the group delay τ of the conventional and the multirate filter.

Filter orders. From the MATLAB output: decimation / interpolation filter $N_{Dec,Int} = 20$, kernel filter $N_{Kernel} = 116$ (both including the safety margin of 2), compared to $N = 391$ for the single-rate filter. The approximation from the lecture, $N_{FIR} \approx \frac{2}{3}\log_{10}\left(\frac{3}{10\,\delta_{pass}\delta_{stop}}\right)\frac{1}{b}$, gives 19.2 and 115.9 and confirms these values.

Computational effort. The effort is the filter order times the sampling frequency at which the filter is computed. The single-rate filter runs at f_s:

$$E_{total,single} = N \cdot f_s = 391 \cdot 50\,\text{kHz} = 19.55 \cdot 10^6 \text{ multiplications/s}.$$

In the multirate system all three filters are computed at f_s/M:

$$E_{total,multi} = (2N_{Dec,Int} + N_{Kernel}) \cdot \frac{f_s}{M} = (2 \cdot 20 + 116) \cdot 12.5\,\text{kHz} = 1.95 \cdot 10^6 \text{ multiplications/s}.$$

The multirate filter therefore needs about ten times fewer operations.

Real-time feasibility. With the assembly routine ($16 + 6N$ cycles per call), the worst-case interrupt, in which the kernel filter (117 coefficients) and one polyphase each of the decimation and interpolation

filter (6 coefficients) are computed, needs about $718 + 2 \cdot 52 \approx 820$ cycles, i.e. about 9 % of the 9120 cycles available per sample (single-rate: about 26 %). A real-time implementation of the multirate filter is therefore very likely.

Group delay. In samples at f_s, the single-rate filter has $\tau_{single} = N/2 = 195.5$ samples (3.9 ms). In the multirate system, the kernel filter runs at f_s/M, so each of its delays counts M times:

$$\tau_{multi} = \frac{N_{Dec,Int}}{2} + M\,\frac{N_{Kernel}}{2} + \frac{N_{Dec,Int}}{2} = 20 + 4 \cdot 58 = 252 \text{ samples } (5.0\,\text{ms}).$$

The multirate filter reduces the computational effort considerably, but has a larger group delay.